

TELPERION: An Untrusted Certificate Compiler for Kernel-Checked, Independently Re-Verified Mathematics

Peter W. Murphy*

Draft — August 27, 2026

Abstract

We present TELPERION, a tool that turns families of concrete mathematical statements into machine-checked Lean 4 theorems by *exact-arithmetic certificate and kernel verification*. The design separates a large, replaceable *generator* from a small *trusted core*: the generator searches for and serializes a certificate (a sum-of-squares decomposition, a Positivstellensatz or Farkas combination, a Wilf–Zeilberger pair, a Chvátal–Gomory derivation, and roughly thirty other shapes), and Lean 4’s kernel re-derives the theorem from that certificate. A defective certificate is therefore a *compile failure*, never a false theorem; the generator is untrusted by construction, and the same pipeline proves an unrelated textbook inequality and a research-scale result through identical machinery.

Beyond kernel acceptance, TELPERION’s output can be handed to an *independent judge*: OpenAI’s COMPARATOR exports each proof and checks that it proves exactly the intended statement, uses only a whitelist of axioms, and survives replay by a second kernel (**nanoda**, an independent Rust reimplementaion) in addition to Lean 4’s own. A soundness bug would have to fool two independently written kernels.

As a flagship demonstration we report the state of an attack on the Brualdi–Goldwasser tree Laplacian-ratio maximization problem—a live question, a 2025 conjectured maximizer having been refuted in 2026. We are explicit about what this demonstrates: the maximizer is *not* determined. What is established is (i) an exact-rational result narrowing the problem—a branching tree provably beats the published spider counterexamples, so the maximizer is not the path-spine spider it was recently conjectured to be—and (ii) a Lean 4 formalization, verified by both kernels and axiom-clean, of a *conditional* reduction of a candidate’s global optimality to two precisely stated open layers, together with the two analytic cruxes it rests on. The contribution we emphasize is exactly this: a methodology in which the boundary between what is proved and what is open is enforced by the kernel and an independent second kernel, and is therefore impossible to overstate.

Draft scope. This is a standalone paper about TELPERION. A separate paper will present a theorem on the Laplacian-ratio maximizer if and when the two open layers are closed; here the work appears only as an in-progress case study, scoped honestly (§5).

Disclosure. TELPERION and the accompanying formalization were developed with substantial assistance from AI coding agents. This is not incidental to the paper’s thesis: the trust model is designed precisely so that the generator — whether a human, a solver, or an AI — is untrusted, and soundness rests only on Lean 4’s kernel and an independent second kernel. See §2 and **[TODO: finalize disclosure wording with author]**.

***[TODO: affiliation, email, ORCID]**. Correspondence: **[TODO: email]**.

1 Introduction

Two anxieties attend the growing use of machines in mathematical proof. The first is *trust*: as large language models, SAT/SMT solvers, and semidefinite programming (SDP) tools increasingly produce candidate proofs and lemmas, how do we know their output is correct, and not merely plausible? The second is a *gap*: many powerful proof techniques—sum-of-squares (SOS) certificates, Positivstellensatz representations, linear-programming duals—are discovered numerically, in floating point, and a numerically “verified” certificate is not a proof. Rounding a near-feasible SDP solution to something a proof assistant will accept, without silently changing the claim, is itself delicate work.

TELPERION addresses both by committing to a strict division of labor. A large, freely replaceable *generator* searches for a certificate and serializes it in exact rational arithmetic. A small *trusted core*—Lean 4’s kernel, and nothing else—re-derives the theorem from that certificate. The generator is never trusted: if it emits a wrong certificate, the emitted Lean 4 fails to compile; it can never yield a false theorem. This is not a slogan but a structural property of the pipeline, and it is what lets the same tool prove Bernoulli’s inequality and drive a research-scale formalization through identical machinery.

Concretely, one describes a problem as a *certifiable family*: a grid of instances, each reducible to one of TELPERION’s certificate *shapes*. The tool certifies each instance in exact arithmetic, then emits Lean 4 that MATHLIB’s kernel re-proves from scratch. Roughly thirty shapes are supported today—rational-function positivity by Pólya lifting, exact SOS (reaching interior equality cases), Artin-denominator rational SOS for nonnegative-but-not-SOS polynomials, Putinar and Handelman certificates on semialgebraic sets, Farkas / linear-Positivstellensatz combinations, Nullstellensatz and real-Nullstellensatz ideal-membership certificates, Wilf–Zeilberger hypergeometric identities, Chvátal–Gomory integer rounding, and more (§3)—and where a statement needs a new shape, one adds an *emitter*; the trust model comes for free.

An independent second opinion. Kernel acceptance already rules out false theorems. But a single implementation—Lean 4’s elaborator and kernel—is still a single point of trust, and kernel acceptance alone cannot certify that the emitted proof proves the statement one *meant*, rather than a vacuous or weakened variant. TELPERION’s output can therefore be handed to an independent judge: the COMPARATOR [9] from OpenAI’s *ten-proofs* project [11]. Given a *challenge* module (the intended statement) and a *solution* module (the emitted proof), the COMPARATOR exports both and checks three things no ordinary build can: that the solution proves *exactly* the challenge statement; that it uses only a declared whitelist of axioms; and that it survives replay not only by Lean 4’s kernel but by **nanoda**—an independent reimplement of the kernel in Rust [1]. A soundness bug would have to fool two independently written kernels (§4).

A flagship, honestly scoped. We use an in-progress attack on the Brualdi–Goldwasser tree Laplacian-ratio maximization problem as the paper’s running research example—a live question in which a 2025 conjectured maximizer was refuted in 2026. We state plainly, and repeatedly, what it demonstrates. The maximizer is *not* determined. What is established is, first, an exact-rational narrowing—a branching tree provably beats the published spider counterexamples—and, second, a Lean 4 formalization of a *conditional* reduction of a candidate’s global optimality to two precisely stated open layers, with the two analytic inequalities that reduction rests on; the latter are sorry-free, axiom-clean, and—as §5 reports—accepted by both Lean 4’s kernel and **nanoda**. The point is not a completeness claim; it is that the boundary between proved and open is here a machine-checked object, not a matter of a reader’s charity toward prose.

Contributions. This paper makes no claim to a new decision procedure or a new certificate class. Its contribution is an *integrated system* and the methodology it embodies:

1. a certificate-compilation pipeline (§3) that turns exact-arithmetic certificate families into kernel-checked Lean 4, with a composable library of ~ 30 emitter shapes and an enforced `certify` $\rightarrow$ `validate` $\rightarrow$ `emit` $\rightarrow$ `freeze` workflow;
2. a trust model (§2) in which the generator is untrusted by construction and the sole trusted component is the kernel, with explicit guards against the one failure the kernel cannot catch—vacuity;
3. an independent-verification layer (§4) that adds statement-identity, an axiom whitelist, and a *second, non-Lean 4 kernel* on top of kernel acceptance; and
4. an end-to-end demonstration on both closed textbook problems and an in-progress research formalization (§5), with the proved/open boundary machine-enforced.

TELPERION is a standalone, problem-agnostic artifact; nothing about the engine is specific to the case study. It is available as open source (§8).

2 The trust model

TELPERION’s design rests on a single principle: **the generator is untrusted, and the Lean 4 kernel is the sole trusted component.** A defective certificate manifests as a compile failure, never as a false theorem. Everything else in the tool—the search heuristics, the SDP rounding, the emitters, even the exact-arithmetic self-checks—exists to make the tool *useful*, not to make its output *true*. Truth is established by, and only by, the kernel re-deriving the theorem from the emitted certificate.

This has a design corollary worth stating: because nothing the generator does is trusted, the generator is free to be large and heuristic while the trusted surface stays small. The engine is a few thousand lines of dependency-light Python (it uses only `sympy`); a referee who wishes to can audit the engine, but need not—the audit that matters is the kernel’s, and it happens on every build. This is the same discipline that underlies certificate-checking in optimization (e.g. VIPR for mixed-integer programs [4]) and the “de Bruijn criterion” in interactive theorem proving [2]: a small trusted checker re-validates the output of an arbitrarily complex, untrusted producer.

The self-checks are a convenience, not the guarantee. Before emitting, TELPERION verifies the certificate numerically in exact arithmetic (`fractions.Fraction` and `sympy.Rational`; no floating point on any certificate path). This catches mistakes cheaply—before a continuous-integration round-trip—but establishes nothing about truth. It is important to be precise here, because it is a common category error: an exact-arithmetic check that “the certificate reproduces the target” is a check of the *generator’s* internal consistency. Only the kernel’s re-derivation is evidence about the theorem.

The one thing the kernel cannot catch: vacuity. The kernel rejects a *false* theorem. It happily accepts a *true-but-vacuous* one: a reflexive $X = X$ or $0 \leq 0$ compiles green while proving nothing about the certificate, because the defect lives in the *statement*, not the proof. TELPERION therefore turns the tool on its own output. The emit step refuses a reflexive emitted statement, and the identity-style emitters additionally require the certificate to be *load-bearing*: a corrupted certificate must break the claim, or emission is refused. A family that deliberately emits a reference identity or a degenerate boundary case opts out explicitly, declaring that intent. Two further local gates run before any build is attempted: a structural lint (unfilled template holes, unbalanced

delimiters, duplicate names) and a *soundness lint* that refuses the “green build $\neq$ proved” classes—a `sorry` or `admit`, a smuggled `axiom`, an empty `:= by`, a missing type ascription, a `Prop := True` stub.

Provenance and drift. Emitted files are stamped with the tool version and a SHA-256 hash over a canonical serialization of every instance’s expressions, the Lean 4 profile, the proof templates, and the emitters’ own code (timestamps excluded). A regeneration diff detects any drift byte-for-byte, so a change to emission logic can never ship under a stale hash, and a hand-edit to a generated file is flagged. There is no API path from a family definition to Lean 4 text that bypasses certification, and emission is refused without both a certified-family witness and a green validation report.

What kernel acceptance still does not give you. Two gaps remain, and they motivate §4. First, kernel acceptance rests on a *single* implementation of the kernel (and the elaborator that produced the proof term); an implementation bug, however unlikely, is a common-mode failure. Second, the non-vacuity guards above are TELPERION checking its *own* output for a narrow syntactic class of degeneracy; they are not an independent confirmation that the emitted proof proves the statement a third party *intended*. Both gaps are closed—to the extent any finite process can—by handing the proof to an independent judge that re-derives the intended statement’s identity, restricts the admissible axioms, and replays the proof through a second, independently written kernel. That is the subject of §4; we first describe how certificates become Lean 4 at all.

3 Certificate families and emitters

TELPERION turns a problem into Lean 4 when it can be expressed as a *certifiable family*—a grid of instances, each reducible to one of a fixed set of certificate *shapes*. This section describes the pipeline, the shapes, and a fully worked example.

3.1 The pipeline

Every family flows through an enforced, four-stage workflow:

```
define → certify() → validate → emit() → lake build → freeze().
```

`define` states the family: its symbols, the grid of parameter values, and a target expression per grid point. `certify()` searches for and checks a certificate for each instance in exact arithmetic, returning a certified-family witness or naming the first cell (and, where relevant, the corner) that failed. `validate` runs the family’s own exact-numeric assertions. `emit()` renders the certified family into Lean 4 through the emitters; it refuses without both the certified-family witness and a green validation report, and it applies the non-vacuity and soundness gates of §2. `lake build`—the user’s own continuous integration—is the trusted step: MATHLIB’s kernel re-derives every emitted theorem. `freeze()` records the output and its provenance hash for drift detection.

The proof text each emitter writes is deliberately narrow and uniform: identities are discharged by `simp only` → `push_cast` → `field.simp` → `ring`, nonnegativity by `positivity`, finite case analysis by `interval.cases`, and so on. The generator does not emit clever proofs; it emits certificates and lets a small, fixed repertoire of kernel-backed tactics re-check them.

3.2 The certificate shapes

The shapes fall into a few broad families; Appendix A gives the full catalog. Positivity and inequalities are covered by rational-function positivity via Pólya lifting (multiply through by $(1 + \sum x_i)^N$ and reduce to a nonnegative-coefficient form), exact rational SOS (which reaches interior equality cases that Pólya lifting cannot), Artin-denominator rational SOS for polynomials that are nonnegative but not SOS (TEPPERION finds a positive multiplier q with $q \cdot p$ a sum of squares), Putinar and Handelman certificates on semialgebraic and polytope constraints, Bernstein positivity on an interval, and Sturm-sequence root exclusion for strict positivity. Exact algebra is covered by identity and p -adic-valuation emitters, Farkas / linear-Positivstellensatz cone combinations, Nullstellensatz and real-Nullstellensatz ideal-membership certificates (cofactors computed by Gröbner reduction), and SOS-refutation certificates of semialgebraic infeasibility. Combinatorial and integer shapes include Wilf–Zeilberger hypergeometric-sum identities, Chvátal–Gomory integer rounding (VIPR-style), Newton/interlacing inequalities, tangent-line convex-sum inequalities, Cauchy–Schwarz, and explicit PSD quadratic forms. Finally, assembly shapes—finite case dispatch, region subdivision and gluing, $\forall K \geq K_0$ tails, substitution glue, and custom skeletons—compose the atomic shapes into statements in the original variables.

Where a numeric solver is involved (finding a Putinar multiplier by SDP, say), the solver runs untrusted: its floating-point solution is rounded to exact rationals and *re-verified exactly* before emission, and only the exact certificate reaches Lean 4. The SDP is a search oracle, not part of the trusted chain.

3.3 A worked example

Take Bernoulli’s inequality in integer form: for every integer $k \geq 2$ and real $x \geq 0$,

$$(1 + x)^k - 1 - kx \geq 0.$$

Expanding, $(1 + x)^k = \sum_{j=0}^k \binom{k}{j} x^j$; the degree-0 and degree-1 terms cancel against $-1 - kx$, leaving

$$(1 + x)^k - 1 - kx = \sum_{j=2}^k \binom{k}{j} x^j,$$

a polynomial with nonnegative integer coefficients over $x \geq 0$. That is exactly a Pólya certificate: the emitter rewrites the target into its nonnegative-coefficient form and closes each instance by **positivity**. Driving the grid $k \in \{2, \dots, 6\}$ through the pipeline yields five Lean 4 theorems of the form

```
theorem bernoulli_k3 (x : R) (hx : 0 <= x) :
  0 <= 3 * x ^ 2 + x ^ 3 := by
  ... -- rewrite to the certified form, then ‘positivity’
```

The example is deliberately humble, but it exercises the *general* core (not any problem-specific machinery), and it makes a soundness point concrete: the $k = 1$ instance is the vacuous $0 \leq 0$, and TEPPERION excludes it—Bernoulli’s inequality is nontrivial only for $k \geq 2$. Harder shapes behave identically from the caller’s view: the Motzkin polynomial $x^4y^2 + x^2y^4 - 3x^2y^2 + 1$, the canonical nonnegative-but-not-SOS example, is handled by the Artin-denominator rational-SOS emitter, which finds a positive multiplier turning it into a sum of squares; the caller sees the same **certify** $\rightarrow$ **emit** interface and a kernel-checked theorem at the end.

4 Independent verification

Kernel acceptance rules out false theorems, but rests on one implementation and says nothing about whether the proof proves the intended statement. TELPERION closes both gaps by submitting its output to an independent judge—the COMPARATOR [9], a tool from OpenAI’s *ten-proofs* release [11] that adjudicates Lean 4 proofs.

4.1 What the Comparator checks

The COMPARATOR takes two Lean 4 modules: a *challenge* (the intended statement) and a *solution* (the proof under test). It compiles both in a sandbox, exports both to a kernel-level term format with `lean4export` [10], and asserts three properties that an ordinary `lake build` does not:

1. **Statement identity.** The solution proves *exactly* the challenge statement—not something weaker. When the challenge is authored apart from the certificate, this is a generator-*independent* form of the vacuity guard: a drifted or hollowed emission fails the type match against a reference statement the generator never touched.
2. **Axiom whitelist.** A per-theorem `#print axioms`, admitting only the axioms one lists—the clean set is `[propext, Quot.sound, Classical.choice]`—and rejecting anything else. This is stronger than a source grep for `sorry`: it rejects a smuggled axiom, and it rejects the `ofReduceBool` axiom that `native.decide` introduces, not merely `sorryAx`.
3. **Kernel replay.** The exported proof is re-checked by Lean 4’s kernel and, with `nanoda` enabled, by `nanoda` [1]—a from-scratch reimplement of the Lean 4 kernel in Rust. A soundness bug must fool two independently written kernels.

The third property is the one that most exceeds ordinary peer review. A human referee reads the prose and, at best, trusts that the accompanying Lean 4 compiles; they do not re-run the kernel, let alone a second one. Here a second, unrelated kernel re-derives the proof term from first principles, and the axiom trace is checked mechanically. The result is a stronger warrant than a referee can personally supply.

4.2 The bridge

TELPERION ships a small bridge module that turns an emit result into a COMPARATOR challenge configuration and scaffolds the challenge module: it extracts the emitted theorems’ fully qualified names, writes the JSON configuration (challenge and solution modules, theorem names, permitted axioms, whether to enable `nanoda`), and—for the case where the statement *can* be re-stated independently—renders a challenge module that restates the emitted signatures and proves them by a tactic independent of TELPERION’s certificate. Sharded (multi-file) emissions receive one configuration per shard. A fully worked, continuous-integration-green example (a MATHLIB build, both kernels, and the end-to-end run) accompanies the tool.

4.3 Two modes, and an honest limit

There are two ways to use the judge, and they differ in strength.

Independent challenge (the stronger mode): the challenge module restates the theorem and proves it *without* TELPERION’s certificate. Statement identity then has real force—a drifted emission fails the type match against a reference the generator did not produce.

Self-check (`challenge_module = solution_module`): used when a statement genuinely cannot be re-stated independently—for instance a capstone whose statement is *about* the project’s own definitions, so any restatement must import and reuse them. Here statement identity is trivial, but

the axiom whitelist and the second-kernel replay remain fully independent verification. We are explicit about this limit in the case study of §5, where the capstone is verified in self-check mode: the second kernel and the axiom trace are real; independent statement authorship is not available for a statement phrased in the formalization’s own vocabulary.

4.4 A note on the sandbox

The COMPARATOR runs its children under a sandbox. For verifying one’s *own* output the sandbox is not the security boundary—the kernel replay is the guarantee—so in that setting a lightweight wrapper suffices. For adjudicating *untrusted third-party* solutions, a real sandbox is warranted, and we provide a bubblewrap-backed wrapper that isolates the filesystem and network while preserving the exporter’s argument conventions. The distinction matters for anyone who would use the COMPARATOR as a public judge of submitted proofs, as opposed to a self-check on a pipeline one already controls.

5 Flagship case study: the Brualdi–Goldwasser Laplacian-ratio problem

We use an in-progress attack on the Brualdi–Goldwasser tree-maximization problem as the paper’s research-scale demonstration. The area is live and moving fast—a 2025 conjecture about the maximizer was refuted in 2026—which makes it an unusually good test of the methodology’s central promise: keeping a machine-checked boundary between what is proved and what is merely believed. We state plainly what is and is not established. **The maximizer is not determined here**; what is established is a new structural narrowing of the problem (verified in exact arithmetic) and a conditional formalization of a candidate’s global optimality (verified by two kernels), with the gap between them explicit.

5.1 The problem and its recent history

Brualdi and Goldwasser [3] introduced the study of the permanent of the Laplacian of a tree and, among other results, asked for the tree on n vertices maximizing the *Laplacian ratio*

$$\pi(T) = \frac{\text{per}(L(T))}{\prod_v \deg(v)}, \quad L = D - A.$$

The maximizer is open. Wu, Dong and Lai [14] conjectured it to be a subdivided star. Pant [12] refuted that conjecture in 2026 with families of *spider* trees $T(a_1, \dots, a_m)$ (a path spine whose vertices carry length-two cherries) that beat the conjectured subdivided star; Pant proposes no replacement, and the true maximizer remains open. This is the state of the art the formalization engages.

5.2 An exact-arithmetic narrowing: branching beats spiders

The project’s first contribution is computational but *exact*—rational arithmetic throughout, no floating point, so “beats” is a strict rational inequality with no margin. It shows the maximizer is not a path-spine spider at all. Concretely, for a branching family $B(3, t)$ —a degree-three hub whose four centers each carry t cherries, on $n = 8t + 4$ vertices—one has, for *every* integer $t \geq 3$,

$$\pi(B(3, t)) - \pi(T(t, t, t, t)) = \left(\frac{3}{2}\right)^{4t} \cdot \frac{2P(t)}{81(t+1)^3(t+2)^2(t+3)}, \quad P(t) = 56t^4 + 72t^3 - 162t^2 - 432t - 243,$$

and $P(t) > 0$ for all $t \geq 3$ (the prefactor and denominator are positive, and P is increasing on $[3, \infty)$ with $P(3) > 0$). Since $T(t, t, t, t)$ is exactly Pant’s flagship counterexample family, this proves *no member of that family is the maximizer*: the maximizer is a branching tree, not a path-spine spider. This is a genuine advance on the published state, and it is the kind of exact-certificate computation TELPERION is built for. Two independent permanent engines (a tree matching-sum recursion and Ryser’s formula on the full Laplacian) agree on the values, an exhaustive search confirms the true maximizer for small n (where it *is* a spider, exhibiting a genuine path-to-branch transition as n grows), and the pipeline reproduces Pant’s published rational values exactly.

We are careful about the verification *tier* here: this narrowing is an exact-rational computation, machine-checked in the project’s certified arithmetic, but it is a computation, not (yet) a Lean4 kernel proof. It motivates a candidate maximizer—a branching-backbone family—whose *global* optimality is the subject of the formalization below.

5.3 The conditional formalization and its two open layers

Establishing that the candidate is the global maximizer— $\forall t, \text{Aobj}(t) \leq \text{Aobj}(\text{tieU})$ for the candidate’s value tieU —is the hard, open part. The formalization’s spine is a single Lean4 theorem, `R3Cert.Step3.conjecture1_of_layers`, which is *conditional*: it derives global optimality from two hypotheses, and is itself sorry-free. Verbatim, it takes a tie witness `tieU` and

```
Hnorm : forall t : UTree, exists s : List Hub,
  Balanced s /\ Capped s /\ Aobj t <= Aobj (backboneU s)
Hdom   : forall s : List Hub, Balanced s -> Capped s ->
  (forall u, not (OrderedStep s u)) -> Aobj (backboneU s) <= Aobj tieU
```

and concludes `forall t : UTree, Aobj t <= Aobj tieU`. The two hypotheses are the open layers: H_{norm} (every tree is dominated by a balanced, capped canonical hub-state) and H_{dom} (every merge-normal such state is dominated by the tie value). Both remain open; a single-hub within-one balancing argument inside H_{norm} is formalized and machine-checked, but the pieces that would close the layer—pinning the extremal arm rates to the specific $\{4, 5\}$ profile, the capping invariant, the tree-to-hub reduction, and multi-hub extremality—are research-open. The end-goal predicate `conjecture1_proved` is defined to be `False` and asserted so by a test.

TELPERION supplies the concrete certificate-bearing pieces of the reduction: the exact-rational positivity floors of the per-class ledger, the per-rule reduction certificates of the internal argument, and the two analytic inequalities the reduction rests on, each an instance of the shapes of §3. Two of those inequalities are the anchor theorems: `R3Cert.phi_le_one`, the bound $\log \Phi(b) \leq 0$ (i.e. $\Phi \leq 1$) for every branch, and `R3Cert.CappedJointConfig.gstep_le_one_achievable`, the g-step / master inequality for achievable configurations.

The model-to-object caveat. The analytic bounds are proved about an exact-rational cavity/matching *model* (written `Aobj`). Identifying this model with $\pi(T)$ itself is a separate step that is not yet formalized in Lean4, and we flag it rather than eliding it. The model’s numerics are cross-checked against the exact integer Laplacian permanent by the two independent engines above, which is evidence—but not a formal proof—of the identification.

5.4 The proved/open ledger

Table 1 is the honest summary, with the verification tier named for each row.

Result	Tier	Meaning
Branching $B(3, t)$ beats Pant’s spider $T(t, t, t, t)$, all $t \geq 3$ <code>conjecture1_of_layers</code>	Exact rational (com- putational) Lean 4 kernel + nanoda	The published counterexample family is non-maximal; the maximizer is a branching tree. Global optimality of the candidate <i>follows from</i> $H_{\text{norm}}, H_{\text{dom}}$. Conditional.
<code>phi.le.one</code> ($\log \Phi \leq 0$)	Lean 4 kernel + nanoda	The $\Phi \leq 1$ crux, on the cavity model.
<code>gstep.le.one.achievable</code>	Lean 4 kernel + nanoda	The g-step / master-inequality crux.
$H_{\text{norm}}, H_{\text{dom}}$	Open	The two layers the reduction is conditional on.
Model-to-object identification	Open	Identifying the cavity model with $\pi(T)$.
Global maximizer	Open	<code>conjecture1.proved = False</code> .

Table 1: The Laplacian-ratio work, by verification tier. The exact-rational narrowing and the Lean 4 anchors are established; global optimality is open.

5.5 Machine-verification evidence

The three Lean 4 anchors are the theorems the formalization’s continuous integration guards, and they are re-verified through the COMPARTOR (§4) on every change. For each, `lean4export` exports the proof, the axiom set is checked against the clean whitelist, and the exported term is replayed by both Lean 4’s kernel and **nanoda**. In our runs all three pass: both kernels accept each, and each reports the axiom trace `[propext, Quot.sound, Classical.choice]`—no `sorryAx`, no `native_decide`. The conditional capstone’s dependency cone is large but tractable: export and two-kernel replay complete in a few minutes of continuous-integration time. Because these statements are phrased in the formalization’s own vocabulary, the COMPARTOR runs them in self-check mode (§4): the axiom whitelist and the independent second kernel are real verification; independent statement authorship is not available for a statement about the project’s own structures, and we do not claim it.

The resulting claim is precise and unfakeable. We do not assert the global maximizer. We assert: (i) in exact arithmetic, a branching tree beats the published spider counterexamples, narrowing the open problem; and (ii) in Lean 4, verified by two independent kernels with a clean axiom trace, a conditional reduction of the candidate’s global optimality to two stated layers, together with the two analytic cruxes it rests on. Everything else—the two layers, the model-to-object identification, and hence global optimality itself—is open, and the boundary is enforced mechanically rather than asserted in prose. A separate paper will report a theorem on the maximizer only if and when that boundary moves.

6 Related work

Interactive theorem proving and its libraries. TELPERION targets Lean 4 4 [5] and MATH-LIB [13]; its trust argument is the long-standing de Bruijn criterion [2]—a small trusted kernel re-checks the output of untrusted producers. What TELPERION adds at this level is not a new foundation but a disciplined *producer*: a certificate compiler whose emitted proofs use only a narrow, fixed repertoire of kernel-backed tactics, so that the object the kernel checks is always a certificate re-derivation rather than an opaque tactic script.

Learned provers. Recent systems—AlphaProof [7], DeepSeek-Prover [6], and related LLM-driven approaches—*search* for proofs, typically as tactic sequences, guided by learned models. TELPERION occupies a different point in the design space: it does not search proof *text*; it computes a *certificate* (often via an untrusted numeric solver) and emits a fixed proof skeleton that re-checks it. The two are complementary—a learned system could propose the family or the certificate shape, and TELPERION could compile and independently verify the result—and the independent-verification layer of §4 is exactly the kind of external check that machine-proposed proofs most need.

Tactic-level automation and Positivstellensatz methods. Lean 4 and its neighbors provide `positivity`, `polyrith`, `nlinarith`, and related tactics; the formal-methods literature includes substantial work on sum-of-squares and Positivstellensatz certificates and their verified checking [8]. TELPERION’s contribution relative to these is integrative rather than a single new procedure: it packages exact certificate *search-and-rounding* for roughly thirty shapes behind one enforced pipeline, with provenance, non-vacuity gates, and independent re-verification.

Certificate checking in optimization. The separation of an untrusted solver from a trusted, exact checker is standard in optimization—VIPR [4] provides checkable certificates for mixed-integer programming, and TELPERION’s Chvátal–Gomory emitter is VIPR-flavored. TELPERION carries that discipline into general theorem proving: the numeric solver is a search oracle, and only an exact, kernel-re-checkable certificate crosses into the trusted region.

Independent proof judging. The independent-verification layer builds directly on the COMPARATOR [9], `lean4export` [10], and `nanoda` [1]. Our contribution there is the *bridge*—turning TELPERION’s emit results into COMPARATOR challenges automatically—and the demonstration that a research-scale proof’s anchors survive a second, independent kernel with a clean axiom trace.

7 Limitations and scope

We are deliberate about what TELPERION is not, because a tool whose purpose is trustworthiness must be honest about its own boundaries.

It proves certifiable families, not arbitrary theorems. TELPERION applies when a statement can be cast as a grid of instances, each reducible to one of its certificate shapes—concrete inequalities, identities, valuation facts, finite case analyses, and their assemblies. It is not a general-purpose prover and does not attempt open-ended proof search; a statement outside the shape library requires a new emitter (which the trust model then covers for free) or does not fit.

Vacuity is guarded, not eliminated. The kernel cannot detect a true-but-vacuous statement, and TELPERION’s non-vacuity checks catch a specific syntactic class (reflexive statements, non-load-bearing certificates). An independently authored challenge module (§4) strengthens this to a generator-independent statement-identity check—but only when the statement can be re-stated without reusing the generator’s own definitions. Where it cannot (the capstone of §5), self-check mode gives the axiom whitelist and the second kernel but not independent statement authorship, and we say so.

The independent check has its own trusted surface. The second-kernel guarantee rests on `lean4export`, the `COMPARATOR`, and `nanoda`. These are independent of TELPERION’s generator—which is the point; a common-mode bug in Lean 4’s own kernel would have to be shared by an unrelated Rust implementation to pass—but they are not themselves free of trust. What the layer buys is the reduction of a single point of failure to the intersection of two independent ones, not its elimination.

Scale. Exporting and re-checking a large proof’s dependency cone is not free: the Brualdi–Goldwasser capstone exports to a substantial term file and takes minutes of the second kernel’s time. This is affordable as a continuous-integration gate on a set of anchor theorems, but exporting every lemma of a large library on every commit would not be, and we do not.

The case study is incomplete. As §5 states, the Laplacian-ratio maximizer is not determined; two layers (H_{norm} , H_{dom}) and a model-to-object identification remain open. What is established—an exact-rational narrowing of the problem, and a two-kernel-verified conditional reduction—demonstrates the methodology and the auditability of a partial result, not a resolution of the problem.

8 Conclusion and availability

TELPERION is a certificate compiler for mathematics that takes trust seriously by giving away as little of it as possible. The generator—human-, solver-, or AI-driven—is untrusted; a small kernel re-derives every theorem from an exact certificate; and the output can be handed to an independent judge that checks statement identity, restricts the axioms, and replays the proof through a second, independently written kernel. The result is a pipeline in which correctness is a mechanical property, and in which—as the Brualdi–Goldwasser case study shows—the line between what is proved and what remains open is itself machine-checked, and so cannot be quietly overstated.

We have argued that this integrated system, rather than any single component, is the contribution. The emitter library, the enforced pipeline, the non-vacuity gates, and the independent second kernel are individually modest; together they make “a machine found this and you don’t have to trust the machine” a precise and testable claim.

Future work. The obvious next step for the case study is to close H_{norm} and H_{dom} and to formalize the model-to-object identification; if that succeeds, the Brualdi–Goldwasser result will be reported as a theorem in a separate paper, with the same two-kernel warrant applied to the unconditional statement. On the tool side, natural directions include broadening the independent-challenge mode to more families, additional emitter shapes, and coupling TELPERION to learned systems that propose families or certificate shapes for it to compile and independently verify.

Availability. TELPERION is open source. **[TODO: repository URL, release tag / DOI, and archived artifact.]** The engine is licensed under the Business Source License 1.1 (research use free; converting to Apache-2.0 after the stated change date); the mathematical content and pre-split snapshots are Apache-2.0. **[TODO: confirm license terms and any CLA before publication.]** The worked examples, the `COMPARATOR` bridge and configurations, and the continuous-integration workflows that produce the verification results reported here are included in the repository, so every claim in §3.3 and §5.5 is reproducible.

Acknowledgments. [TODO: acknowledgments. Note the AI-assistance disclosure from the front matter and any funding/collaborators.]

A Emitter catalog

Table 2 lists the certificate shapes (“emitters”) available in the version described here. Each flows through the same `certify` $\rightarrow$ `validate` $\rightarrow$ `emit` $\rightarrow$ `freeze` workflow; “finds” marks shapes where TELPERION searches for the certificate (typically via an untrusted numeric solver, rounded and re-verified exactly) rather than requiring it as input.

Emitter	Proves
Direct Pólya	$0 \leq f(\bar{x})$, f rational with nonnegative-numerator / positive-factored-denominator form.
Bilinear box	before $\leq$ after on a box in two bound variables, via four Pólya corner certificates.
SOS	$0 \leq p$ via an exact rational PSD-Gram sum of squares (reaches interior ties).
Rational SOS (Artin)	$0 \leq p$ for nonnegative-but-not-SOS p : finds $q > 0$ with $q \cdot p$ a sum of squares (e.g. Motzkin).
Constrained SOS (Putinar)	$0 \leq p$ on $\{g_i \geq 0\}$; supply or <i>find</i> the SOS multipliers.
Handelman	$0 \leq p$ on a polytope via nonnegative products of the constraints; supply or <i>find</i> .
Bernstein	$0 \leq p$ on $[a, b]$ via nonnegative Bernstein coefficients (found by degree elevation).
Sturm-positive	$0 < p$ (strict) on $[a, b]$ by Sturm root exclusion plus a Bernstein margin.
Cone/Farkas	$0 \leq \text{target}$ as a nonnegative combination of a positivity-provable basis (linear Positivstellensatz).
Nullstellensatz	$p = 0$ on a variety via ideal-membership cofactors (Gröbner).
Real-Nullstellensatz	$p = 0$ on a real variety via $p^{2^m} + \sigma \in \langle g_k \rangle$.
SOS-refutation	semialgebraic infeasibility over $\mathbb{R}$; supply or <i>find</i> the refutation.
Infeasibility	a polynomial system has no common zero (Nullstellensatz refutation $1 = \sum \lambda_j g_j$).
Wilf–Zeilberger	hypergeometric/binomial sum identities via a WZ mate.
Chvátal–Gomory	integer linear goals via CG rounding (VIPR-style; discharged by omega).
Interlacing	Newton’s inequalities of a real-rooted polynomial.
Tangent-sum	symmetric convex-sum inequalities via the tangent line (surplus is an exact SOS).
Cauchy–Schwarz	weighted Cauchy–Schwarz / QM–AM via the pairwise-difference SOS.
PSD form	$0 \leq x^\top M x$ for an explicit rational PSD matrix (completing the square).
Identity / exact fact	integer/rational identities, powers, closed forms (ring/norm_num).
p -adic valuation	$v_p(n) = k$ as decidable divisibility.
Interval bracket	rigorous two-sided rational enclosure $lo \leq e^{-\theta} \leq hi$.

Emitter	Proves
Unimodal / log-concave	the integer max of a (log-)concave sequence at its crossing point.
Telescoping potential	a recursive/tree bound $\sum \text{local}(v) \leq P(\text{root})$ from per-node super-solutions.
Lattice box	$f(x) \leq B$ on $\mathbb{Z}_{\geq 0}^d$ (integer Positivstellensatz).
Assembly shapes	finite case dispatch, subdivision-and-glue, $\forall K \geq K_0$ tails, reparametrization, substitution glue, dichotomy glue, custom skeletons.

Table 2: Certificate shapes. **[TODO: reconcile against the released version’s exact list and count before publication.]**

B Verified anchors of the case study

The three Lean 4 theorems re-verified through the COMPARATOR in §5.5, with their exact signatures:

```
-- R3Cert.Step3.conjecture1_of_layers (conditional capstone)
theorem conjecture1_of_layers (tieU : UTree)
  (Hnorm : forall t : UTree, exists s : List Hub,
    Balanced s /\ Capped s /\ Aobj t <= Aobj (backboneU s))
  (Hdom : forall s : List Hub, Balanced s -> Capped s ->
    (forall u, not (OrderedStep s u)) -> Aobj (backboneU s) <= Aobj tieU) :
  forall t : UTree, Aobj t <= Aobj tieU

-- R3Cert.phi_le_one (the Phi <= 1 crux, on the cavity model)
theorem phi_le_one (b : Branch) : logPhi b <= 0

-- R3Cert.CappedJointConfig.gstep_le_one_achievable (g-step / master inequality)
theorem gstep_le_one_achievable (l : List Q) (hl : forall mu in l, Achievable mu) :
  (baseOf l) ^ 11 * prodBcap l / (W * (5 / 3) ^ 11) <= 1
```

Each reports axiom trace [propext, Quot.sound, Classical.choice] under both Lean 4’s kernel and nanoda—no sorryAx, no ofReduceBool. **[TODO: optionally paste the verbatim #print axioms lines from an AxiomGuard.lean CI run for the record.]**

C Reproducing the verification

The verification results are produced by continuous integration and are reproducible locally. In outline (see the repository workflows for exact, pinned commands; all components are pinned to the same Lean 4 toolchain, v4.32.0):

1. Build the formalization: `lake exe cache get` then `lake build`.
2. Build the judge from the matching tag: `lake build lean4export comparator` in a checkout of `comparator@v4.32.0`; build `nanoda` with `cargo build --release`.
3. Run each anchor: `lake env comparator <anchor>.comparator.json` from the project directory, with the exporter, `nanoda`, and `sandbox` binaries on the path.

A green run reports, per anchor, that both kernels accept the exported proof and that its axiom set is within the whitelist.

References

- [1] ammkrn. nanoda: a Lean type-checker in Rust. https://github.com/ammkrn/nanoda_lib. Accessed 2026.
- [2] Henk Barendregt and Freek Wiedijk. The challenge of computer mathematics. *Philosophical Transactions of the Royal Society A*, 363(1835):2351–2375, 2005. Source for the de Bruijn criterion (a small trusted checker).
- [3] Richard A. Brualdi and John L. Goldwasser. Permanent of the Laplacian matrix of trees and bipartite graphs. *Discrete Mathematics*, 48(1):1–21, 1984.
- [4] Kevin K. H. Cheung, Ambros Gleixner, and Daniel E. Steffy. Verifying integer programming results. In *Integer Programming and Combinatorial Optimization (IPCO 2017)*, volume 10328 of *Lecture Notes in Computer Science*, pages 148–160. Springer, 2017.
- [5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [6] DeepSeek-AI. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. arXiv:2504.21801, 2025.
- [7] Google DeepMind AlphaProof and AlphaGeometry teams. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. **[TODO: confirm the author list of record and volume/pages]**.
- [8] John Harrison. Verifying nonlinear real formulas via sums of squares. In *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, volume 4732 of *Lecture Notes in Computer Science*, pages 102–118. Springer, 2007.
- [9] Lean FRO. Comparator: a trustworthy judge for Lean proofs. <https://github.com/leanprover/comparator>. Accessed 2026; tag v4.32.0 used here. **[TODO: confirm author-ship line]**.
- [10] Lean prover community. lean4export: export Lean 4 environments to a kernel-level term format. <https://github.com/leanprover/lean4export>. Accessed 2026.
- [11] OpenAI. ten-proofs: formalized proofs accompanying “ten advances in mathematics and theoretical computer science”. <https://github.com/openai/ten-proofs>. Accessed 2026. **[TODO: confirm the companion paper’s exact title/citation]**.
- [12] Priyanshu Pant. Counterexamples to a conjecture on Laplacian ratios of trees. arXiv:2605.14176, 2026. **[TODO: confirm the full author list]**.
- [13] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.
- [14] Tingzeng Wu, Xiangshuai Dong, and Hong-Jian Lai. Two problems on Laplacian ratios of trees. *Discrete Applied Mathematics*, 372:224–236, 2025.